

MILESTONE 3

RL Agent Implementation & Training

Game Optimization using Deep Reinforcement Learning

Project	Game AI using DQN and Tabular RL
Phase	Implementation & Initial Training Runs
Target Games	Pac-Man (DQN), Snake (Refined DQN), Flappy Bird (Q-Learning/SARSA)
Implementation Framework	PyTorch, OpenAI Gym, Google Colab
Persistence	Google Drive (models, logs, checkpoints, videos)
Status	Implementation Phase - Focus on Correctness & Reproducibility

Executive Summary

Milestone 3 transitions from theoretical architecture to live implementation. Three paper-aligned RL agents are deployed on their respective environments: Pac-Man (DQN with CNN + dueling head), Snake (refined DQN with training gap strategy), and Flappy Bird (tabular Q-Learning and SARSA). Each agent learns through repeated gameplay episodes using temporal-difference updates. The training loop is: (1) Reset environment, (2) Epsilon-greedy action selection, (3) Execute action, (4) Receive reward & next state, (5) Store in replay buffer (DQN agents), (6) Sample minibatch, (7) Compute TD loss, (8) Backprop & optimizer step, (9) Periodically sync target network, (10) Log results. All training runs are seeded for reproducibility. Checkpoints save every 100 episodes. Success is measured by: agents learn faster than random baselines, learning curves smooth and increasing, no NaN/Inf in losses, annotated gameplay shows learned behaviors. Milestone 3 prioritizes correctness over performance optimization. Once all three agents demonstrate learning, checkpoints are frozen for Milestone 4 (rigorous comparison and analysis).

1. Implementation Roadmap (Phases 3.1–3.9)

Phase	Task	Deliverable	Success Criterion
3.1	Agent Architecture	3 RL agents (DQN×2, Q-Learning, SARSA)	All agents instantiate without error; parameter counts logged
3.2	State Processing	Preprocessing pipeline for all 3 games	Tensor shapes correct (84×84×4, 64×64×4, 5D); no NaN values
3.3	Action Selection	ϵ -Greedy with decay schedule	Exploration rate decreases over time; actions sampled correctly
3.4	Learning Loop	Core training loop: state→action→reward→update	Loop runs without crash; reward signal received
3.5	DQN Training	Experience replay, minibatch sampling	Replay buffer fills; TD loss computed; networks sync
3.6	Flappy Training	Q-Learning and SARSA update rules	Q-values update per episode; both algorithms run in parallel
3.7	Logging	CSV logs: episode, reward, loss, epsilon	Files, Q-values to Drive; data parseable; no missing values
3.8	Initial Eval	Compare trained vs. random agent	Trained agent score >> random; learning curve visible
3.9	Debugging	Verify reward signals, state transitions	All 10 on-flight checklist passed; training stable

2. Core Training Loop Overview

```
FOR episode in range(num_episodes):
    env.reset()
    state ← get_initial_state()
    episode_reward = 0
    episode_loss = []

    WHILE not done:
        # 1. Action selection
        if random() < epsilon: action ← random
        else: action ← argmax(Q(state))

        # 2. Environment step
        reward, next_state, done ← env.step(action)
        episode_reward += reward

        # 3. Store & update
        replay_buffer.add(state, action, reward, next_state, done)

        IF replay_buffer.size > batch_size:
            batch ← replay_buffer.sample(batch_size)
            loss = compute_td_loss(batch)
            optimizer.step(loss)
            episode_loss.append(loss)

        # 4. Periodically sync target network
        IF total_steps % target_update_freq == 0:
            target_network ← online_network

    state ← next_state

# 5. Logging & decay
```

```
log_episode(episode, episode_reward, episode_loss, epsilon)
epsilon = max(epsilon_min, epsilon * decay)

IF episode % checkpoint_freq == 0:
    save_checkpoint(model, replay_buffer, optimizer)
```

3. Pac-Man DQN: Complete Architecture & Configuration

3.1 Input Preprocessing:

Pac-Man environment outputs RGB frames (84×84). Preprocessing: (1) Convert to grayscale; (2) Normalize to [0, 1]; (3) Stack last 4 frames along channel dimension → (4, 84, 84) tensor. This stacking provides temporal context (enables agent to infer velocity/motion).

```
class PacManDQN(nn.Module):
    def __init__(self, num_actions=4):
        super().__init__()
        # Input: (batch, 4, 84, 84)

        # Convolutional backbone (learns visual features)
        self.conv1 = nn.Conv2d(4, 32, kernel_size=3, stride=1, padding=1)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1)
        self.conv3 = nn.Conv2d(64, 64, kernel_size=3, stride=1, padding=1)
        # Output after conv3: (batch, 64, 84, 84)

        # Flatten: 64 * 84 * 84 = 451,584 features
        self.fc = nn.Linear(64*84*84, 512)

        # Dueling architecture: separate V(s) and A(s,a)
        self.value_stream = nn.Linear(512, 1)
        self.advantage_stream = nn.Linear(512, num_actions)

    def forward(self, state):
        x = F.relu(self.conv1(state))
        x = F.relu(self.conv2(x))
        x = F.relu(self.conv3(x))
        x = x.view(x.size(0), -1) # Flatten
        x = F.relu(self.fc(x))

        # Dueling output
        value = self.value_stream(x) # (batch, 1)
        advantage = self.advantage_stream(x) # (batch, num_actions)

        # Combine: Q(s,a) = V(s) + (A(s,a) - mean(A(s,a')))
        Q_values = value + (advantage - advantage.mean(dim=1, keepdim=True))
        return Q_values # (batch, num_actions)
```

Component	Value	Rationale
Input shape	(4, 84, 84)	4 stacked frames; 84×84 from original
Conv layers	32→64→64 filters	Progressive feature refinement; 3×3 kernels, stride=1
FC layer	512 hidden units	Sufficient capacity for feature aggregation
Dueling split	Value + Advantage	Wang et al. 2016: decouples state quality from action choice
Num actions	4 (up, down, left, right)	Pac-Man discrete action space
Replay buffer	100,000 transitions	Large enough for diverse experience; manageable memory
Batch size	32	Balances gradient stability & computational efficiency
Learning rate	0.00025	Mnih et al. 2015 baseline; small for stability

Gamma (γ)	0.99	High discount; values rewards 100 steps ahead
Target update	Every 1000 steps	Provides stable bootstrap targets
ϵ start/end	1.0 $\rightarrow$ 0.01	Full exploration $\rightarrow$ exploitation; 500k frame decay
Reward	+0.1 pellet, +0.5 power, -0.01 step, -0.5 pellet	Onset of pellet collection; penalizes ghost collision & dawdling

4. Snake: Refined DQN with Training Gap & Dual Replay

4.1 Problem Context (Wei et al. 2018):

Snake exhibits extreme sparsity: rewards appear only when apple eaten. Between apples, all transitions provide zero reward signal, making learning difficult. Solution: (1) **Reward shaping**: Add +0.1 survival reward to guide exploration, (2) **Training gap**: Skip the K frames immediately after food (sudden state change adds noise), (3) **Dual replay**: Prioritize high-error transitions.

```
class SnakeRefinedDQN(nn.Module):
    def __init__(self, num_actions=4):
        super().__init__()
        # Lighter CNN for 64x64 input
        self.conv1 = nn.Conv2d(4, 16, kernel_size=3, stride=1, padding=1)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, stride=1, padding=1)
        self.conv3 = nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1)
        self.fc = nn.Linear(64*64*64, 256)

        # Dueling head
        self.value = nn.Linear(256, 1)
        self.advantage = nn.Linear(256, num_actions)

    def forward(self, state):
        x = F.relu(self.conv1(state))
        x = F.relu(self.conv2(x))
        x = F.relu(self.conv3(x))
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc(x))
        value = self.value(x)
        advantage = self.advantage(x)
        Q = value + (advantage - advantage.mean(dim=1, keepdim=True))
        return Q

class DualReplayBuffer:
    def __init__(self, high_capacity=35000, low_capacity=15000):
        self.high_td_buffer = deque(maxlen=high_capacity) # High error
        self.low_td_buffer = deque(maxlen=low_capacity) # Low error

    def add(self, transition, td_error):
        if td_error > threshold:
            self.high_td_buffer.append(transition)
        else:
            self.low_td_buffer.append(transition)

    def sample(self, batch_size):
        n_high = int(batch_size * 0.7) # 70% from high-error
        n_low = batch_size - n_high # 30% from low-error
        high_batch = random.sample(self.high_td_buffer, n_high)
        low_batch = random.sample(self.low_td_buffer, n_low)
        return high_batch + low_batch
```

Component	Value	Rationale
Input shape	(4, 64, 64)	Smaller input→faster training; 4 stacked frames
Conv layers	16→32→64 filters	Lighter than Pac-Man; sufficient for snake grid
FC layer	256 hidden units	Adequate capacity; reduces overfitting

Dueling split	Value + Advantage	Separates state value from action choice
Replay buffer	Dual (35k high + 15k low)	High-error buffer: recent challenging experiences
TD threshold	0.5	Transitions with $ TD_error > 0.5 \rightarrow$ high buffer
Batch size	32	32 total: 70% high (22), 30% low (10)
Learning rate	0.0001	Smaller than Pac-Man; avoids instability
Gamma (γ)	0.99	Same as Pac-Man
Target update	Every 500 steps	More frequent sync due to training gap
Training gap	K=4 frames	Skip 4 transitions after apple eaten
ϵ schedule	1.0 $\rightarrow$ 0.02	Decay over 200k frames; more exploration needed
Reward	+1.0 apple, +0.1 survival, -1.0 death	Well-balanced design; sparsity mitigated by survival reward

5. Flappy Bird: Q-Learning & SARSA (Tabular)

5.1 State Discretization (FlapAI Bird 2020):

Unlike DQN agents, Flappy Bird uses tabular RL operating on discretized state. Continuous observations (bird_y, pipe_x, pipe_gap) are binned. State = (x_diff_bin, y_diff_bin, y_vel_bin) $\in \{0-4\} \times \{0-4\} \times \{0-2\}$, giving $5 \times 5 \times 3 = 75$ total states. Q-table is Python dict {state: {action: value}}. Two algorithms run in parallel: Q-Learning (off-policy) and SARSA (on-policy).

```
def preprocess_flappy_state(bird_y, bird_x, pipe_x, gap_center, y_vel):
    """Convert continuous state to discrete bins."""

    # Relative features
    x_diff = pipe_x - bird_x # Distance to next pipe
    y_diff = gap_center - bird_y # Distance to gap center
    # y_vel already in game units (-10 to 10)

    # Discretize: divide into bins
    x_bin = min(4, max(0, (x_diff + 50) // 50)) # Bin 0-4
    y_bin = min(4, max(0, (y_diff + 60) // 30)) # Bin 0-4
    v_bin = min(2, max(0, (y_vel + 10) // 10)) # Bin 0-2

    return (x_bin, y_bin, v_bin)

class FlappyQTable:
    def __init__(self):
        self.Q = {} # State → Action → Q-value
        self.alpha = 0.1 # Learning rate
        self.gamma = 0.99 # Discount
        self.epsilon = 1.0 # Exploration

    def ensure_state(self, state):
        if state not in self.Q:
            self.Q[state] = {0: 0.0, 1: 0.0} # 0=do_nothing, 1=flap

    def get_Q(self, state, action):
        self.ensure_state(state)
        return self.Q[state][action]

def q_learning_update(agent, s, a, r, s_prime, done):
    """Off-policy Q-Learning: use max Q of next state."""
    agent.ensure_state(s)
    agent.ensure_state(s_prime)

    if done:
        Q_target = r
    else:
        max_Q_next = max(agent.Q[s_prime].values())
        Q_target = r + agent.gamma * max_Q_next

    Q_old = agent.Q[s][a]
    agent.Q[s][a] = Q_old + agent.alpha * (Q_target - Q_old)

def sarsa_update(agent, s, a, r, s_prime, a_prime, done):
    """On-policy SARSA: use actual next action."""
    agent.ensure_state(s)
    agent.ensure_state(s_prime)

    if done:
        Q_target = r
    else:
        Q_target = r + agent.gamma * agent.Q[s_prime][a_prime]
```

```

Q_old = agent.Q[s][a]
agent.Q[s][a] = Q_old + agent.alpha * (Q_target - Q_old)

```

Component	Q-Learning	SARSA	Rationale
State space	75 discrete states	75 discrete states	Manageable for tabular method
Q-table	Dict {state: {0: q0, 1: q1}}	Dict {state: {0: q0, 1: q1}}	Fast lookup; memory efficient
Learning rate α	0.1	0.1	Standard for tabular RL
Discount γ	0.99	0.99	Consistent with DQN agents
ϵ start/end	1.0 $\rightarrow$ 0.05	1.0 $\rightarrow$ 0.05	Decay over 5000 episodes
Update rule	$Q(s,a) += \alpha(r + \gamma \max_{a'} Q(s',a') - Q(s,a))$	$Q(s,a) += \alpha(r + \gamma Q(s',a') - Q(s,a))$	Off-policy vs. on-policy
Next action	Best Q-value	Actual ϵ -greedy sampled	Bootstraps best vs. actual
Reward	+0.5 frame, +5 pipe pass, -1000 collision	+0.5 frame, +5 pipe pass, -1000 collision	Incentivize survival & milestones
Episodes	5000–10000	5000–10000	Adequate for convergence

6. Action Selection & Epsilon-Greedy Exploration

```
def epsilon_greedy_action(Q_values, epsilon, num_actions=None):
    """Choose action: explore (random) or exploit (greedy)."""
    if random.random() < epsilon:
        # Explore: random action
        if num_actions:
            return random.randint(0, num_actions - 1)
        else:
            return random.choice(list(Q_values.keys()))
    else:
        # Exploit: greedy (best Q-value)
        return max(Q_values, key=Q_values.get)

# Epsilon decay schedule
def decay_epsilon(epsilon, decay_rate=0.9999, epsilon_min=0.01):
    return max(epsilon_min, epsilon * decay_rate)

# Training loop with epsilon decay
epsilon = 1.0
for episode in range(num_episodes):
    state = env.reset()
    episode_reward = 0

    while not done:
        # Get Q-values for current state
        if isinstance(agent, DQNAgent):
            Q_values = agent.network(state) # Neural network
            action = epsilon_greedy_action(Q_values, epsilon, num_actions)
        else: # Tabular agents
            Q_dict = agent.Q[state]
            action = epsilon_greedy_action(Q_dict, epsilon)

        reward, next_state, done = env.step(action)
        agent.update(state, action, reward, next_state, done)
        episode_reward += reward
        state = next_state

    epsilon = decay_epsilon(epsilon) # Decrease exploration
    log_metrics(episode, episode_reward, epsilon)
```

7. Logging and Metrics

Every episode, log to CSV: (episode, cumulative_reward, episode_length, loss, epsilon, avg_Q). For DQN agents, also track: minibatch_loss, target_sync_events. For Flappy Bird, track: Q_table_size (number of visited states). Save logs to Google Drive every 100 episodes.

```
class Logger:
    def __init__(self, filepath):
        self.filepath = filepath
        self.data = []

    def log_episode(self, episode, reward, length, loss, epsilon, avg_Q=None):
        record = {
            'episode': episode,
            'cumulative_reward': reward,
            'episode_length': length,
            'avg_loss': loss,
```

```

        'epsilon': epsilon,
        'avg_Q': avg_Q or 'N/A'
    }
    self.data.append(record)

def save_csv(self):
    df = pd.DataFrame(self.data)
    df.to_csv(self.filepath, index=False)
    print(f"Logged {len(self.data)} episodes to {self.filepath}")

# Usage
logger = Logger('training_logs.csv')
for episode in range(num_episodes):
    episode_reward = train_episode(...)
    avg_loss = compute_avg_loss(...)
    logger.log_episode(episode, episode_reward, episode_length, avg_loss, epsilon)

if episode % 100 == 0:
    logger.save_csv() # Persist to Drive

```

8. Model Checkpointing and Recovery

```
def save_checkpoint(agent, optimizer, replay_buffer, episode, filepath):
    """Save model state for resumable training."""
    checkpoint = {
        'episode': episode,
        'model_state': agent.state_dict(),
        'optimizer_state': optimizer.state_dict(),
        'replay_buffer': replay_buffer,
        'timestamp': datetime.now().isoformat()
    }
    torch.save(checkpoint, filepath)
    print(f"Saved checkpoint: {filepath}")

def load_checkpoint(filepath):
    """Resume training from checkpoint."""
    checkpoint = torch.load(filepath)
    return checkpoint['model_state'], checkpoint['optimizer_state'], checkpoint['replay_buffer'],
    checkpoint['episode']

# Training with checkpointing
checkpoint_freq = 100 # Save every 100 episodes
for episode in range(start_episode, num_episodes):
    train_episode(...)

    if episode % checkpoint_freq == 0:
        save_checkpoint(agent, optimizer, replay_buffer, episode, f'checkpoint_ep{episode}.pt')

# For Flappy Bird (tabular)
def save_qtable(q_table, filepath):
    import pickle
    with open(filepath, 'wb') as f:
        pickle.dump(q_table, f)

def load_qtable(filepath):
    import pickle
    with open(filepath, 'rb') as f:
        return pickle.load(f)
```

9. Initial Evaluation Protocol

```
def evaluate_agent(agent, env, num_episodes=100, render=False):
    """Test trained agent ( $\epsilon=0$ : greedy only, no exploration)."""
    rewards = []
    lengths = []

    for episode in range(num_episodes):
        state = env.reset()
        episode_reward = 0
        episode_length = 0

        while not done:
            # Greedy action only (no exploration)
            action = agent.get_best_action(state)
            reward, next_state, done = env.step(action)
            episode_reward += reward
            episode_length += 1
            state = next_state
```

```

        if render:
            env.render()

        rewards.append(episode_reward)
        lengths.append(episode_length)

    # Compute statistics
    mean_reward = np.mean(rewards)
    std_reward = np.std(rewards)
    mean_length = np.mean(lengths)

    print(f"Mean Reward: {mean_reward:.2f} ±{std_reward:.2f}")
    print(f"Mean Episode Length: {mean_length:.2f}")

    return {'mean_reward': mean_reward, 'std': std_reward, 'mean_length': mean_length}

# Baseline: random agent
def evaluate_random_agent(env, num_episodes=100):
    rewards = []
    for _ in range(num_episodes):
        state = env.reset()
        ep_reward = 0
        while not done:
            action = env.action_space.sample() # Random action
            reward, _, done = env.step(action)
            ep_reward += reward
        rewards.append(ep_reward)
    return np.mean(rewards), np.std(rewards)

```

10. Phase 3.9: Debugging and Validation Checklist

✓	Item	What to Check	Success Signal
1	Reward Signal	Does agent receive correct reward for game events?	Flappy: +5 on pipe, -1000 on collision. Log shows expected values.
2	State Shape	Are state tensors/tuples the correct shape?	DQN: (4, 84, 84) or (4, 64, 64). Flappy: (5,) tuple. No NaNs.
3	Normalization	Are pixel values normalized consistently?	DQN: values in [0, 1]. Grayscale: uint8 → float32/255.
4	Terminal Flag	Does 'done' trigger correctly at game end?	Game over → done=True. Episodes don't run forever.
5	Replay Buffer	Is buffer filling correctly? No corrupted transitions?	Buffer size increases; oldest entries evicted (ring buffer).
6	TD Loss	Is loss computed without NaN/Inf? Does it decrease?	Loss starts high, decreases smoothly. No NaN/Inf.
7	Target Network	Does target network sync occur without backlogs?	Target copied from online; no grad through target.
8	Epsilon Decay	Does epsilon decrease as expected?	Starts high (1.0), ends low (0.01–0.05). Smooth decay.
9	Checkpoint Save/Load	Can we save and resume training?	Load checkpoint; metrics match previous run (±0.1%).
10	Trained > Random	Does trained agent vastly outperform random?	Trained reward >> random. Improvement > 500%.

11. Milestone 3 Deliverables (Complete Checklist)

#	Deliverable	Format	Purpose	Location
1	Three functioning game environments	Python modules	Wrap OpenAI Gym; handle preprocessing	envs/
2	Pac-Man DQN model	PyTorch .pt	Trained agent for Pac-Man	models/pacman_dqn_best.pt
3	Snake Refined DQN model	PyTorch .pt	Trained agent for Snake	models/snake_dqn_best.pt
4	Flappy Bird Q-Learning Q-table	Pickle .pkl	Tabular Q-function for Flappy	models/flappy_q_table.pkl
5	Flappy Bird SARSA Q-table	Pickle .pkl	Tabular SARSA function for Flappy	models/flappy_sarsa_table.pkl
6	Training script (Pac-Man)	Jupyter .ipynb	03_train_pacman.ipynb	notebooks/
7	Training script (Snake)	Jupyter .ipynb	03_train_snake.ipynb	notebooks/
8	Training script (Flappy)	Jupyter .ipynb	03_train_flappy.ipynb	notebooks/
9	Pac-Man reward log	CSV	episode, reward, length, loss, epsilon, Q-table	logs/pacman_training.csv
10	Snake reward log	CSV	episode, reward, length, loss, epsilon, Q-table	logs/snake_training.csv
11	Flappy reward log	CSV	episode, reward, length, epsilon, Q-tables	logs/flappy_training.csv
12	Pac-Man loss curve	PNG	matplotlib plot: episode vs. loss	plots/pacman_loss.png
13	Snake loss curve	PNG	matplotlib plot: episode vs. loss	plots/snake_loss.png
14	Pac-Man learning curve	PNG	episode vs. cumulative reward	plots/pacman_learning.png
15	Snake learning curve	PNG	episode vs. cumulative reward	plots/snake_learning.png
16	Flappy learning curve	PNG	episode vs. cumulative reward (Q-Learning & SARSA)	plots/flappy_learning.png
17	Pac-Man trained gameplay	MP4	Annotated video of trained agent	videos/pacman_trained.mp4
18	Snake trained gameplay	MP4	Annotated video of trained agent	videos/snake_trained.mp4
19	Flappy trained gameplay	MP4	Annotated video for Q-Learning & SARSA	videos/flappy_trained.mp4
20	Pac-Man vs Random comparison	PNG	Side-by-side bar chart: trained vs. random	plots/pacman_comparison.png
21	Snake vs Random comparison	PNG	Side-by-side bar chart: trained vs. random	plots/snake_comparison.png
22	Flappy vs Random comparison	PNG	Side-by-side bar chart: trained vs. random	plots/flappy_comparison.png
23	Pac-Man hyperparameters	JSON	config_pacman.json: all settings logged	configs/
24	Snake hyperparameters	JSON	config_snake.json: all settings logged	configs/
25	Flappy hyperparameters	JSON	config_flappy.json: all settings logged	configs/
26	Training metadata	JSON	Seed, package versions, timestamp	metadata.json
27	Reproducibility report	TXT	Instructions to reproduce all results	REPRODUCIBILITY.txt

12. Success Criteria & Transition to Milestone 4

Category	Criterion	Metric / Signal
Convergence	All agents learn (reward increases)	Training curve: smooth upward trend after initial episodes
Performance	Trained >> random baseline	Improvement > 500%; e.g., trained=100, random=15
Stability	No divergence or NaN losses	Loss curve: smooth, no Inf/NaN; training completes without crash
Reproducibility	Results repeatable with fixed seed	Run training 3×: all produce $\pm 0.1\%$ identical metrics
Exploration	Epsilon decays correctly	ϵ starts at 1.0, ends at 0.01–0.05; smooth decay over training
Checkpointing	Save/load cycle works	Save checkpoint, load, resume; episode count and loss match
Logging	Metrics recorded accurately	CSV files have no missing values; data matches run output
Behavior	Learned behaviors visible in video	Trained agent avoids obstacles, collects rewards (gameplay shows strategy)

Key Philosophy for Milestone 3:

" **1. Correctness over Performance:** Focus on getting agents to learn AT ALL, not on state-of-the-art scores. " "A simple, correct DQN that learns is better than an overfitted one that doesn't.

" **2. Reproducibility First:** Fix all random seeds (torch, numpy, random). Save configs. Document hyperparameters. Future runs must produce $\pm 0.1\%$ identical results.

" **3. Separation of Concerns:** Training loop separate from evaluation loop. No data leakage. No touching test set during training.

" **4. Iterative Debugging:** If agent doesn't learn, check in this order: (1) Reward design, (2) State representation, (3) Hyperparameters, (4) Algorithm. Don't change algorithm first.

" **5. Documentation as You Go:** Log everything. Annotate code. Save metadata. You should be able to re-run any experiment in 5 minutes from saved configs and checkpoints."

Milestone 4 Preview:

Once Milestone 3 checkpoints are frozen, Milestone 4 will: (1) **Standardize evaluation**—all agents tested on same 1000 episodes, " "same seed; (2) **Compute rigorous metrics**—ROC-AUC, PR-AUC, success rate, latency; (3) **Perform ablations**—disable reward shaping, " "change network size, vary hyperparameters; (4) **Analyze failure modes**—when and why agents fail; (5) **Answer research questions**—Does DQN outperform " "Q-Learning? When? Why? (6) **Write final conclusions**—paper-quality analysis and comparison. Milestone 4 is the analysis phase; Milestone 3 is the engineering phase."

Immediate Next Steps (To Begin Milestone 3):

1. **Set up environment wrappers**—Create OpenAI Gym wrappers for all 3 games with preprocessing
2. **Implement RL agents**—DQN (Pac-Man), Refined DQN (Snake), Q-Learning & SARSA (Flappy Bird)
3. **Test on small scale**—Run 100 episodes per agent to verify correctness
4. **Debug & fix**—Use 10-point checklist to validate reward, state, terminal conditions
5. **Run full training**—1000–10,000 episodes per agent; save checkpoints every 100 episodes
6. **Generate plots**—Learning curves, loss curves, baseline comparisons
7. **Record gameplay**—Annotated MP4 videos showing trained behavior
8. **Save all outputs**—Models, logs, plots, videos to Google Drive with metadata
9. **Write summary**—Brief report on Milestone 3 results and any issues encountered
10. **Prepare for Milestone 4**—Freeze checkpoints; plan rigorous evaluation protocol